MAHARASHTRA UNIVERSITY
STUDENTS ASSOCIATION
"By the students / For the students / With the students"

MUSA (MUMBAI UNIVERSITY STUDENTS ASSOCIATION)

QUESTION BANK SOLUTION OF SOFTWARE TESTING & QUALITY ASSURANCE

B.E SEM-VII | SCHEME: R-19 C SCHEME | FOR WINTER SESSION 2026

BRANCH: IT | BATCH: REGULAR & KT

Q.1) Explain Agile testing and its life cycle along with the challenges in Agile Testing.

Ans:

1) Agile Testing

1) Agile Testing is a software testing approach in which testing is performed continuously throughout development instead of waiting until the entire software is completed.
2) It follows Agile development principles, where software is developed in small iterations or sprints and working features are delivered frequently.
3) The main goal is to detect defects early, improve software quality and confirm that each feature satisfies changing customer requirements.
4) Developers, testers, business analysts and product owners work closely together during planning, implementation and testing.
5) Testing begins with requirement discussions and continues through coding, integration, acceptance and release.
6) Frequent feedback helps the team understand defects quickly and improve the product in the same or following sprint.
7) Manual and automated testing are used according to the feature, risk and available resources.
8) Regression testing is repeated whenever changes are introduced so previously working features remain reliable.
9) Agile Testing values collaboration, working software and adaptability while still requiring sufficient test evidence.
10) It is commonly practiced in Scrum, Kanban and other iterative software development environments.

Stages of Agile Testing Life Cycle: The Agile Testing Life Cycle consists of five main phases:

1) Impact Assessment:

1) Impact Assessment is the first phase of the Agile Testing Life Cycle, in which the testing team studies the requirements and proposed changes in the software.
2) Testers communicate with developers, business analysts and customers to understand the features that need to be developed or modified.
3) The main objective is to identify how new requirements or changes may affect existing functionality and testing activities.
4) The testing team identifies the modules, features and interfaces that may require additional testing.

2) Agile Testing Planning:

1) Agile Testing Planning is the phase in which the testing team prepares a testing strategy for the upcoming sprint or iteration.
2) Testers and developers work together to understand user stories, acceptance criteria and testing requirements.
3) The testing team identifies which functionalities need to be tested and determines the appropriate testing techniques.
4) Test cases, test data, testing environments and automation requirements are planned according to the sprint objectives.
5) Testing responsibilities are assigned to team members to ensure that all important functionalities are covered.
6) The testing team estimates the time and effort required to complete testing activities within the sprint.

3) Release Readiness:

1) Release Readiness is the phase in which the testing team evaluates whether the developed software is ready for delivery or release.
2) Testers execute the planned test cases to verify that the application satisfies the defined requirements and acceptance criteria.
3) Functional testing, regression testing and other necessary testing activities are performed to identify remaining defects.
4) The testing team reviews the defects discovered during testing and verifies whether critical defects have been resolved.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or is strictly prohibited.

4) Daily Scrums:

1) Daily Scrums are short meetings conducted regularly during Agile development to discuss the progress of development and testing activities.
2) Developers, testers and other team members share updates about their completed work and ongoing tasks.
3) Team members discuss what they completed previously, what they plan to work on next and whether they are facing any difficulties.
4) Testers communicate information about newly identified defects, pending test cases and issues affecting testing progress.
5) Any obstacles, such as unavailable test environments, unclear requirements or unresolved defects, are identified so that the team can coordinate solutions.

5) Test Agility Review:

1) Test Agility Review is the final phase in which the testing team evaluates the effectiveness of testing activities performed during the iteration.
2) Testers review the completed testing work, identified defects and overall testing results.
3) The team examines whether the testing objectives were achieved and whether the selected testing techniques were effective.
4) Problems faced during testing, such as delays, communication gaps or insufficient test coverage, are discussed.

3) Challenges in Agile Testing

1) **Frequently Changing Requirements:** Customer expectations can change during a sprint, so test cases and acceptance criteria may need frequent updates.
2) **Limited Testing Time:** Short sprint durations make it difficult to perform every desirable test before the release deadline.
3) **Continuous Regression Testing:** Repeated code changes require retesting of existing features to detect unintended effects.
4) **Insufficient Documentation:** Lightweight documentation may leave testers unclear about some business rules and expected results.
5) **Communication Challenges:** Weak coordination among testers, developers and customers can cause misunderstandings or missed defects.
6) **Automation Difficulties:** Automated tests need continuous maintenance when interfaces, workflows and data change.
7) **Integration Problems:** New features are integrated frequently, which may expose incompatibilities among modules.
8) **Resource Limitations:** Teams may lack specialist testers, realistic test environments or sufficient testing tools.
9) **Maintaining Test Coverage:** Testers must cover new functionality, existing behavior and important negative scenarios despite limited time.
10) **Quality versus Delivery Speed:** Teams must balance fast delivery with security, reliability, usability and defect correction.

Q.2) Discuss Automation Testing tool selection and cost criteria and explain the need.

Ans:

Automation Testing Tool Selection Criteria

1) **Application Compatibility:** The selected tool must support the application type, technology stack, operating system and programming languages used by the project. A browser-only tool may not be suitable for a desktop application.
2) **Testing Requirements:** The tool should support the required types of testing, such as functional, regression, API or performance testing. A tool is suitable only when it can automate the actual test scenarios.
3) **Ease of Learning and Use:** The interface, scripting approach and documentation should be understandable to the testing team. A difficult tool can increase training time and reduce productivity.
4) **Reusability and Maintainability:** The tool should allow reusable test steps, organized test data and easy updates when the application changes. Otherwise, maintaining scripts may cost more than executing manual tests.
5) **Platform and Browser Support:** The tool should work on the browsers, devices and operating systems used by customers. Cross-platform support is important when the application has many target environments.
6) **Integration with Other Tools:** It should connect with CI/CD systems, version control, defect tracking and test management tools. Integration makes automated testing part of the development process.
7) **Reporting and Debugging:** The tool should provide clear pass/fail results, logs, error messages and other evidence to help testers investigate failed tests.
8) **Execution Speed and Reliability:** The tool should execute tests consistently and support repeated or parallel execution when needed. Unstable scripts produce false failures and waste time.

MUSA

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or u is strictly prohibited.

9) **Availability of Support:** Vendor assistance, community support, training resources and regular updates are important when the team encounters technical difficulties.

10) **Scalability and Future Requirements:** The tool should support a growing number of tests and changing project needs without requiring the organization to replace the entire automation framework.

11) **Proof of Concept:** Before final selection, the team should automate a few representative test cases and compare practical results, maintenance effort and limitations.

Cost Criteria for Selecting Automation Testing Tools

1) **License and Subscription Cost:** Commercial tools may require one-time purchases, annual subscriptions or per-user fees. Open-source tools may avoid license charges but still involve other expenses.

2) **Installation and Setup Cost:** Configuring software, test servers, browsers, drivers and supporting systems requires time and technical resources.

3) **Training Cost:** Testers may need training in scripting, framework design and tool operation before they can create reliable automated tests.

4) **Test Script Development Cost:** Designing a framework, preparing test data and writing scripts require initial effort even when the tool is free.

5) **Test Maintenance Cost:** Scripts must be updated when screens, workflows, locators or requirements change. This is often a major long-term expense.

6) **Infrastructure Cost:** Devices, virtual machines, cloud services and parallel test execution systems may require additional spending.

7) **Support and Upgrade Cost:** Paid support, specialized consultants, tool upgrades and compatibility updates may increase the total cost.

8) **Execution and Operational Cost:** Test execution consumes computing resources and staff time for analyzing failures and managing test environments.

9) **Total Cost of Ownership:** The organization should calculate the complete cost over the tool's expected life rather than considering only its purchase price.

10) **Return on Investment:** The selected tool should provide enough savings through repeated execution, faster feedback and reduced manual effort to justify its cost.

Need for Automation Testing

1) Automation testing reduces repetitive manual work by executing the same test cases automatically whenever software changes.

2) It supports frequent regression testing so previously working features can be checked after bug fixes and new releases.

3) Automated test suites can run during the night or after each build, providing quick feedback to developers.

4) It improves consistency because the same inputs, actions and expected results are applied during repeated executions.

5) It allows teams to test many browser and operating-system combinations more efficiently when supported by the tool.

6) It supports continuous integration and continuous delivery by detecting failures early in the development pipeline.

7) It helps create repeatable execution logs and reports for tracking test outcomes and software quality.

8) It saves time and cost in the long term for stable test cases that must be executed many times.

9) It increases the time available for testers to perform exploratory, usability and other judgment-based testing.

10) Automation cannot replace all manual testing; it is most beneficial when the test cases are stable, repeatable and worth maintaining.

Q.3) Explain McCall's quality factors and criteria.

Ans:

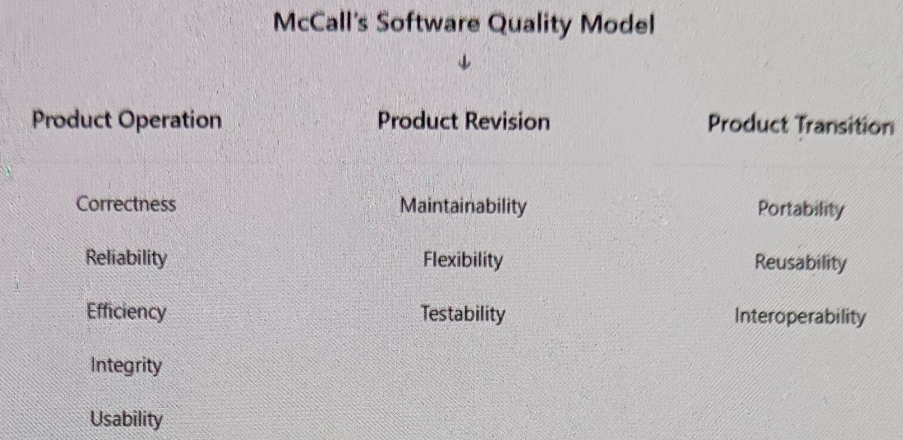
McCall's Software Quality Model:

1) McCall's Software Quality Model was developed by Jim McCall and his colleagues in 1977 to evaluate and improve the quality of software products.
2) It is also known as the McCall Quality Model or General Electric (GE) Quality Model.
3) The model identifies different quality factors that determine how well software satisfies user requirements and performs its intended functions.
4) It divides software quality into three major categories: Product Operation, Product Revision and Product Transition.
5) These three categories contain a total of 11 software quality factors used to evaluate different characteristics of a software product.
6) Each quality factor is supported by measurable or assessable quality criteria that help evaluate the software more systematically.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

7) The model helps software developers, testers and quality assurance teams identify weaknesses and improve software quality.

Classification of McCall's Quality Factors:



1) Product Operation Factors: Product Operation factors describe how effectively software performs its intended functions during normal operation.

A) Correctness:

1) Correctness refers to the ability of software to perform its functions according to specified requirements.
2) It checks whether the software produces accurate results for the given inputs.
3) Software is considered correct when its actual behavior matches the expected behavior defined in the requirements.
4) It helps identify errors in calculations, processing and business logic.
5) Example: A banking application must calculate the correct account balance after every transaction.

B) Reliability:

1) Reliability refers to the ability of software to perform its required functions without failure under specified conditions for a given period.
2) Reliable software produces consistent results during repeated operations.
3) It measures how frequently the software experiences failures during execution.
4) Example: An online banking system should process transactions consistently without unexpected crashes.

C) Efficiency:

1) Efficiency refers to the ability of software to perform its functions using minimum or appropriate system resources.
2) It evaluates the execution speed, response time and resource utilization of software.
3) Efficient software uses CPU, memory and storage resources effectively.
4) It helps improve application performance and reduce unnecessary processing delays.
5) Example: A search engine should display relevant search results quickly without excessive resource consumption.

D) Integrity:

1) Integrity refers to the ability of software to protect programs and data from unauthorized access or modification.
2) It ensures that only authorized users can access sensitive information and perform restricted operations.
3) Integrity is maintained through access controls, authentication and authorization mechanisms.
4) Example: In a banking application, one customer must not be able to modify another customer's account information.

E) Usability:

1) Usability refers to how easily users can learn, understand and operate the software.
2) It evaluates the simplicity of the user interface, navigation and overall user interaction.
3) Software with good usability requires less training and allows users to perform tasks efficiently.
4) Example: An online shopping application should allow customers to search for products and complete purchases easily.

2) Product Revision Factors: Product Revision factors describe how easily software can be modified, corrected and improved after development.

A) Maintainability:

1) Maintainability refers to the ease with which software defects can be identified, corrected and maintained.
2) It determines how quickly developers can locate problems and make necessary changes.
3) Well-structured and properly documented code improves software maintainability.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

4) Example: Developers should be able to fix a login error without modifying unrelated modules.

B) Flexibility:

1) Flexibility refers to the ability of software to accommodate changes in requirements or functionality.
2) It determines how easily existing software can be modified to support new features.
3) Flexible software allows developers to introduce changes without redesigning the complete system.
4) Modular design and well-organized code help improve flexibility.
5) Example: An e-commerce application should allow developers to add a new payment method without major changes to the existing system.

C) Testability:

1) Testability refers to the ease with which software can be tested to verify its correctness and identify defects.
2) Highly testable software allows testers to design and execute test cases easily.
3) Clear inputs, expected outputs and modular components improve testability.
4) Testability helps reduce the effort required for debugging, verification and regression testing.
5) Example: A calculator application with separate functions for addition, subtraction and multiplication can be tested easily.

3) Product Transition Factors: Product Transition factors describe how easily software can be transferred, reused or integrated with other systems.

A) Portability:

1) Portability refers to the ability of software to operate in different hardware or software environments.
2) It determines how easily an application can be transferred from one operating system or platform to another.
3) Portable software requires minimum modifications when moving between supported environments.
Example: An application that can run on Windows, Linux and macOS with minimal changes has good portability.

B) Reusability:

1) Reusability refers to the ability of software components to be used in other applications or systems.
2) It allows developers to use existing modules instead of developing the same functionality repeatedly.
3) Reusable components reduce development time, cost and duplication of work.
4) Example: A login authentication module developed for one application can be reused in another application.

C) Interoperability:

1) Interoperability refers to the ability of software to communicate and exchange information with other software systems.
2) It ensures that different applications can work together effectively.
3) Interoperability is achieved through compatible interfaces, protocols and data formats.
4) Example: An online shopping website communicates with a payment gateway to complete customer transactions.

McCall's Software Quality Criteria

1) Software quality criteria are specific characteristics used to evaluate the quality factors defined in McCall's model.
2) While quality factors describe the overall quality goals, quality criteria describe the properties that contribute to achieving those goals.
3) The model identifies 23 quality criteria that can be associated with one or more quality factors.
4) These criteria help developers and testers assess software quality in a more systematic manner.
5) Important quality criteria include:
    - **Accuracy:** Correctness of calculations and outputs.
    - **Completeness:** Presence of all required functions.
    - **Consistency:** Uniformity in design and operation.
    - **Error Tolerance:** Ability to continue operating when errors occur.
    - **Execution Efficiency:** Efficient use of processing time.
    - **Storage Efficiency:** Efficient utilization of memory and storage.
    - **Access Control:** Protection against unauthorized access.
    - **Operability:** Ease of operating the software.
    - **Simplicity:** Ease of understanding the software structure.
    - **Modularity:** Division of software into independent components.
    - **Self-Descriptiveness:** Ability of software components to explain their purpose through clear structure and documentation.
    - **Expandability:** Ease of adding new functionality.
    - **Instrumentation:** Ability to monitor and measure software operation.
    - **Machine Independence:** Reduced dependence on specific hardware.

MUSA

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

©2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

6) For example, the quality factor Efficiency can be evaluated using execution efficiency and storage efficiency.

7) Similarly, Maintainability can be evaluated using criteria such as simplicity, modularity and self-descriptiveness.

8) Thus, quality criteria provide a more detailed basis for evaluating and improving the 11 quality factors in McCall's Software Quality Model.

Q.4) Explain Regression testing and its types in detail.

Ans:

Regression Testing

1) Regression Testing means executing tests again after a software change to ensure that previously working features still function correctly.
2) Changes may include bug fixes, new functions, code refactoring, configuration updates and modifications to connected modules.
3) The main objective is to detect unintended side effects caused by software modifications.
4) **Regression testing differs from retesting:** retesting checks whether a reported defect was fixed, while regression testing checks whether the fix affected other functions.
5) It is especially important in Agile and continuous delivery environments because new builds are produced frequently.
6) Automated regression tests are useful for stable and repeatedly executed scenarios, but manual regression testing may still be needed.

Types of Regression are:

1) Corrective Regression Testing

1) It is used when the software specifications and requirements have not changed.
2) The existing test cases can be reused without major redesign.
3) Its purpose is to check whether changes such as defect corrections have disturbed existing behavior.
4) It is suitable when the change is small and the expected system behavior remains the same.
5) It saves test design effort because previously prepared tests remain applicable.
6) Example: After fixing a login error, the team runs existing login and account-access tests again.

2) Progressive Regression Testing

1) It is used when software requirements or specifications have changed.
2) Existing test cases are reviewed because some expected results may no longer be valid.
3) New test cases are designed for newly introduced or modified functionality.
4) Previously working functions are also checked for unintended side effects.
5) It is useful when a product is enhanced with additional features or changed business rules.
6) Example: When a shopping application adds a new payment method, the team creates new tests and repeats related checkout tests.

3) Selective Regression Testing

1) Only a selected subset of the available regression tests is executed.
2) The selection is based on changed modules, dependencies and possible impact on other functions.
3) It reduces execution time compared with running every existing test.
4) The team must carefully identify affected areas to avoid missing important defects.
5) It is suitable when testing time is limited and the software change is localized.
6) Example: A profile-page change may require profile, authentication and account-data tests rather than every system test.

4) Complete Regression Testing

1) The team executes the complete regression test suite for the software system.
2) It provides broad checking of existing functionality after major modifications.
3) It is commonly considered before important releases or after large architectural changes.
4) It may discover defects in areas that were not expected to be affected.
5) It requires more execution time, infrastructure and analysis effort than selective testing.
6) Example: After a major platform upgrade, the team reruns all important end-to-end workflows.

5) Partial Regression Testing

©2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

1) It focuses on the modified component and related parts of the application.

2) It checks that the changed module works correctly with connected unchanged modules.

3) The scope is broader than isolated unit retesting but smaller than full-system regression testing.

4) It is useful when a change may affect interfaces or data exchange between components.

5) Test cases are selected from both modified and dependent functionality.

6) Example: A change to order calculation is checked together with checkout and invoice generation.

6) Unit Regression Testing

1) It tests an individual modified unit, function or class after a code change.

2) The unit is checked independently as far as practical using controlled inputs and expected outputs.

3) It detects local side effects before broader integration testing begins.

4) Developers commonly automate these tests in unit testing frameworks.

5) It is fast and useful for frequent code changes but cannot replace integration or system regression testing.

6) Example: After changing a discount function, the developer reruns all tests for that function.

7) Retest-All Regression Testing

1) All existing test cases are rerun without selecting only affected tests.

2) The method is straightforward because it does not require complex test selection.

3) It provides extensive execution of the available suite but does not guarantee that every defect will be found.

4) It is expensive when the suite is large and many tests are repeated unnecessarily.

5) It is suitable when a comprehensive check is required and sufficient time and resources are available.

6) Example: Before a critical release, a team reruns the complete set of stored tests.

Q.5) Explain any two automation tools in detail.

Ans: ***Note: The university examination may ask students to explain any two automation/testing tools. The following syllabus-listed tools are provided so students can prepare any suitable two.***

1) Selenium

1) Selenium is an open-source suite primarily used to automate web browser interactions for testing web applications.

2) Selenium WebDriver allows test scripts to control supported browsers and interact with page elements.

3) Test scripts can be written using supported programming languages such as Java, Python, C# and JavaScript.

4) Testers can automate actions such as opening pages, entering text, clicking buttons and checking displayed results.

5) Selenium Grid supports running browser tests across multiple machines and browser configurations.

6) It is commonly used for functional and regression testing of web applications.

7) Selenium can be integrated with build pipelines and external test frameworks for assertions and reports.

8) Its advantages include flexibility, broad browser support and no Selenium licensing charge.

9) Its limitations include test maintenance when page elements change and lack of direct support for many desktop application interfaces.

2) IBM Rational Functional Tester (RFT)

1) IBM Rational Functional Tester is a commercial tool for automating functional and regression tests in supported application environments.

2) It helps testers automate repeated interactions with application user interfaces.

3) The tool provides recording and scripting capabilities for creating test procedures.

4) Object recognition features help locate and interact with supported interface elements.

5) Verification points can compare actual application behavior with expected results.

6) Automated scripts can be executed repeatedly after software changes to check existing functions.

7) Execution results help testers identify failures and investigate defects.

8) It is useful in enterprise environments that require vendor-supported automation.

9) Advantages include enterprise-oriented features and support options; limitations include licensing cost, training and platform compatibility requirements.

3) JIRA

1) JIRA is primarily a project and issue-tracking tool, not a standalone test execution automation engine.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

2) Testing teams use JIRA to create, assign, prioritize and track software defects.

3) An issue can record steps to reproduce, expected results, actual results, screenshots and severity information.

4) Custom workflows help teams manage defect states such as Open, In Progress, Fixed and Closed.

5) Dashboards and reports help managers monitor testing progress and unresolved defects.

6) JIRA can be connected with test management applications and automation pipelines through integrations.

7) Teams can link defects to requirements, development tasks and releases for better traceability.

8) Its advantages include collaboration and visibility; limitations include dependence on integrations for actual automated test execution.

4) Bugzilla

1) Bugzilla is an open-source bug tracking system used to report and manage software defects.

2) Testers can create bug reports with descriptions, reproduction steps, priority, severity and attachments.

3) Defects can be assigned to responsible team members and tracked through a defined workflow.

4) Search and filtering features help teams find defects by product, status, component or assignee.

5) Notifications keep relevant users informed when a bug report changes.

6) Reports and defect histories help teams monitor quality and investigate recurring problems.

7) Bugzilla supports defect management but does not itself automate browser or application test execution.

8) Its advantages include structured defect tracking and no commercial license fee; limitations include setup and administration effort.

5) TestDirector / HP Quality Center

1) TestDirector, later associated with HP Quality Center, is a test management product used to organize software testing activities.

2) It helps teams manage requirements, test plans, test cases and test execution information.

3) Testers can organize test cases into suites and record actual execution results.

4) Defects can be reported and tracked alongside the testing process.

5) Traceability features help connect requirements with test cases and defects.

6) Managers can review testing progress and quality status through reports.

7) It is useful for coordinating structured testing across teams and releases.

8) It is mainly a test management platform; automated execution depends on compatible integrations and tools.

Q.6) Explain Alpha and Beta Testing in detail. What is the entry exit criteria for Alpha testing?

Ans:

1) Alpha Testing

1) Alpha Testing is an early form of acceptance-related testing performed before a software product is released to external users.

2) It is normally conducted within the development organization in a controlled test environment.

3) Internal testers, developers and sometimes selected internal users perform realistic activities using the application.

4) Its main purpose is to identify important defects in functionality, usability, stability and business workflows.

5) Both planned test cases and exploratory testing may be used to discover unexpected behavior.

6) Alpha Testing helps identify problems that were not detected during unit, integration or system testing.

7) Defects found during the alpha phase are recorded, prioritized, corrected and retested.

8) Alpha Testing may involve several rounds until the software reaches an acceptable level of stability.

9) It helps reduce the risk of exposing serious defects to external users during beta testing.

10) **Example:** An organization tests its new mobile banking application internally before inviting customers to try it.

2) Beta Testing

1) Beta Testing is conducted by selected external users or customers before the general release of a software product.

2) It is performed in real or realistic user environments, often using different devices and network conditions.

3) The main purpose is to gather feedback about actual usability, compatibility, performance and reliability.

4) Users carry out their normal activities rather than following only predefined internal test cases.

5) Beta Testing can reveal environmental problems and user expectations that internal testers did not anticipate.

MUSA

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

6) Feedback may be collected through bug reports, surveys, support channels or application feedback features.

7) The development team reviews feedback, corrects important defects and improves the product before release.

8) Beta Testing provides evidence of product readiness but does not guarantee the software is free from defects.

9) **Example:** A company releases a limited beta version of its application to a selected group of customers.

3) Entry Criteria for Alpha Testing

1) The major features planned for the alpha build should be implemented and available for testing.

2) The application build should be sufficiently stable to install, launch and perform important operations.

3) Basic unit and integration testing should be completed for the included functionality.

4) The alpha testing scope, objectives and expected results should be defined.

5) Test cases, suitable test data and the required testing environment should be prepared.

6) Critical defects that prevent meaningful testing should be resolved or formally accepted as known limitations.

7) The responsible project or test lead should approve the start of alpha testing.

4) Exit Criteria for Alpha Testing

1) The planned alpha test cases should be executed to the agreed level of coverage.

2) Critical and high-priority defects should be corrected and verified or assessed under an agreed exception process.

3) Important functional workflows should meet their acceptance criteria.

4) Regression tests should confirm that defect fixes have not damaged existing functionality.

5) Remaining defects should be documented with severity, impact and a plan for resolution.

6) Test results and identified risks should be reviewed by the responsible team.

7) The team should confirm that the product is ready for beta testing or the next planned stage.

Q.7) Explain Mutation testing in detail.

Ans:

1) Mutation Testing

1) Mutation Testing is a fault-based testing technique used to measure how effectively a test suite detects small programming errors.

2) It creates modified copies of the original program by introducing small intentional changes called mutations.

3) Each modified program is called a mutant, while the original program is called the source program.

4) Mutations are created using mutation operators that change parts of expressions, conditions or statements.

5) The existing test cases are executed against the mutants and their results are compared with those of the original program.

6) A mutant is killed if a test case detects a difference in externally observable behavior.

7) A mutant survives when none of the executed tests detects its changed behavior.

8) An equivalent mutant behaves identically to the original program for all relevant inputs and cannot be killed by any valid test.

9) The technique helps identify weak tests even when statement or branch coverage appears high.

10) Mutation Testing is often used for unit-level code because running many modified programs can be expensive.

2) Common Mutation Operators

1) **Arithmetic Operator Replacement:** Replace operators such as + with − or * with / to check whether tests detect incorrect calculations.

2) **Relational Operator Replacement:** Replace conditions such as > with >= or == with != to test boundary and comparison behavior.

3) **Logical Operator Replacement:** Replace AND with OR or change logical expressions to detect missing condition coverage.

4) **Constant Replacement:** Modify numeric or Boolean constants to test whether expected values are checked correctly.

5) **Statement Deletion:** Remove or skip a statement to determine whether the test suite notices missing program behavior.

3) Steps in Mutation Testing

1) Prepare the original source program and a test suite with expected results.

2) Run the test suite on the original program to confirm that the tests pass.

3) Generate mutants by applying small changes using mutation operators.

4) Execute the test suite separately on each mutant.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

5) Compare the mutant results with the original results and mark detected mutants as killed.

6) Analyze surviving mutants to identify missing inputs or weak assertions.

7) Identify equivalent mutants so they are not incorrectly treated as weaknesses.

8) Improve the test suite and repeat the process if needed.

9) Calculate the mutation score to summarize the effectiveness of the tests.

4) Mutation Score

Mutation Score (%) = (Killed Mutants / Non-equivalent Mutants) × 100

Example: If 18 of 20 non-equivalent mutants are killed, Mutation Score = (18 / 20) × 100 = 90%.

5) Advantages and Limitations

1) It evaluates whether tests detect faults rather than only whether program statements were executed.

2) It encourages better test inputs and more meaningful assertions.

3) It can reveal missing boundary cases and incomplete condition testing.

4) It gives a numerical measure of test suite effectiveness through the mutation score.

5) It can require considerable processing time because many mutants must be executed.

6) Equivalent mutants are difficult to identify and may complicate interpretation of results.

Q.8) Explain Verification in high level and low level design.

Ans:

1) Meaning of Verification

1) Verification checks whether software work products follow the specified requirements, standards and design rules.

2) It asks whether the team is building the product correctly according to the agreed specifications.

3) Verification is commonly performed through inspections, reviews, walkthroughs and static analysis.

4) It helps identify errors in requirements and design documents before implementation begins.

5) High-Level Design and Low-Level Design are verified at different levels of detail.

2) Verification of High-Level Design (HLD)

1) High-Level Design describes the overall architecture, major modules, interfaces and data flow of the proposed system.

2) The design is checked against the software requirement specification to confirm that all major functions are represented.

3) Reviewers verify that the selected architecture is suitable for the required performance, security, reliability and scalability.

4) Major modules are checked for clear responsibilities and appropriate separation of functions.

5) Interfaces between modules and external systems are examined for consistency and completeness.

6) Data flow diagrams and architectural diagrams are reviewed to identify missing connections or incorrect dependencies.

7) Error handling, access control and important system constraints are considered at the architectural level.

8) Design alternatives and technical risks are reviewed to confirm that the proposed architecture is feasible.

9) The design is checked for maintainability and testability so future changes and testing remain manageable.

10) Any missing modules, conflicting interfaces or design weaknesses are documented and corrected before detailed design.

3) Verification of Low-Level Design (LLD)

1) Low-Level Design describes the internal details of each module, including functions, classes, algorithms and data structures.

2) The detailed design is checked against the approved HLD and assigned module requirements.

3) Function names, input parameters, output values and internal interfaces are reviewed for correctness.

4) Algorithms and pseudocode are examined to verify that the intended logic is correct.

5) Conditional statements, loops and decision paths are checked for missing cases or incorrect conditions.

6) Boundary values, invalid inputs and exceptional situations are reviewed to ensure proper handling.

7) Data structures, database operations and data conversions are checked for consistency and accuracy.

8) Module dependencies are examined to confirm that detailed components interact correctly.

9) The design is checked for simplicity, modularity, readability and compliance with project standards.

10) LLD verification supports unit test preparation because the expected behavior of each component becomes clearer.

MUSA

Q.9) Explain in detail Test Point Analysis (TPA) also describe the procedure for TPA.

Ans:

1) Test Point Analysis (TPA)

1) Test Point Analysis is a structured method for estimating the amount of testing effort required for a software application.
2) It uses the functional size of the software as a starting point and considers characteristics that influence testing work.
3) TPA is useful during test planning when the team needs an estimate of testing time, staffing and cost.
4) The method helps relate the size and complexity of the application to the amount of test preparation and execution required.
5) Function Point Analysis is commonly used to estimate functional size before calculating test points.
6) Test points represent an adjusted measure of the testing workload rather than simply the number of individual test cases.
7) The estimated test points can be converted into effort using productivity information or calibration from previous projects.
8) TPA helps managers prepare testing schedules and allocate resources before actual execution begins.
9) Its accuracy depends on the correctness of the functional size estimate and the assumptions used for the project.
10) TPA estimates should be reviewed and updated when project requirements, test scope or available resources change.

2) Procedure for Test Point Analysis

1) **Step 1 — Define the Test Scope:** Identify the software functions, project boundaries and testing activities that must be included in the estimate.
2) **Step 2 — Determine Functional Size:** Estimate the functional size of the application, commonly using Function Point Analysis.
3) **Step 3 — Analyze the Application:** Study the complexity of the functions and the characteristics that affect the amount of testing needed.
4) **Step 4 — Determine the Required Test Quality:** Identify the required depth of testing and quality expectations for the system.
5) **Step 5 — Assess Testing Conditions:** Consider the testing environment, tools, available documentation and team productivity under the selected TPA method.
6) **Step 6 — Calculate Test Points:** Apply the adjustment rules of the chosen TPA model to obtain the test point estimate.
7) **Step 7 — Convert Test Points into Effort:** Use appropriate organizational productivity data to estimate the required testing hours or person-days.
8) **Step 8 — Prepare the Testing Plan:** Allocate the estimated effort to test design, test execution, defect reporting and retesting.
9) **Step 9 — Review the Estimate:** Validate the assumptions and estimates with project stakeholders and revise them when necessary.
10) **Step 10 — Monitor Actual Effort:** Compare the estimate with actual testing work and use the results to improve future estimates.

Note: Exact numerical factors and formulas vary by the prescribed TPA variant. The procedure above explains the method without introducing an unsupported universal formula.

Q.10) What are the key elements of Test management? Explain in detail the structure of Testing Group.

Ans:

1) Test Management is the process of planning, organizing, monitoring and controlling software testing activities to ensure that the software meets the specified quality requirements.
2) It involves managing testing resources, schedules, test cases, defects and testing activities throughout the Software Testing Life Cycle (STLC).
3) The main objective of Test Management is to ensure that testing is completed effectively within the available time, budget and resources.
4) It helps testing teams identify defects, monitor testing progress and maintain the overall quality of the software.
5) Effective Test Management improves coordination between developers, testers, project managers and other stakeholders.

The key elements of Test Management are:

1) Test Planning:

1) Test Planning is the process of defining the objectives, scope, strategy and activities required for software testing.
2) It identifies which software features need to be tested and which testing techniques will be used.
3) It includes preparing test schedules, estimating testing efforts and allocating resources.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

4) A test plan also defines the entry criteria, exit criteria and expected deliverables of testing.

2) Test Organization:

1) Test Organization involves defining the structure of the testing team and assigning responsibilities to its members.
2) It identifies the roles of test managers, test leads, test engineers and automation testers.
3) Proper organization ensures that testing activities are distributed according to the skills and experience of team members.
4) It improves communication, coordination and accountability within the testing team.

3) Test Estimation and Scheduling:

1) Test Estimation determines the approximate time, effort and cost required to perform testing activities.
2) Estimation is based on software complexity, project size, testing requirements and available resources.
3) Test Scheduling defines the starting and completion dates of different testing activities.
4) Proper estimation and scheduling help the testing team complete testing within project deadlines.

4) Test Design and Execution:

1) Test Design involves preparing test cases, test scenarios and test data based on software requirements.
2) Test cases contain input values, execution steps and expected results.
3) During Test Execution, testers run the prepared test cases and compare actual results with expected results.
4) Any differences between actual and expected results are identified and reported as defects.

5) Defect Management:

1) Defect Management is the process of identifying, reporting, tracking and resolving defects found during software testing.
2) Each defect is recorded with details such as description, severity, priority and steps to reproduce the problem.
3) Defects are assigned to developers for correction and are retested after the required changes are made.
4) Effective defect management ensures that important defects are resolved before software release.

6) Test Monitoring and Control:

1) Test Monitoring involves continuously observing the progress and effectiveness of testing activities.
2) Testing progress is measured using metrics such as executed test cases, passed test cases, failed test cases and defect counts.
3) Test Control involves taking corrective actions when actual testing progress differs from the planned schedule.
4) It helps identify delays, resource shortages and other problems that may affect testing objectives.

7) Risk Management:

1) Risk Management involves identifying potential problems that may affect software quality or testing activities.
2) Risks may include insufficient testing time, unavailable resources, changing requirements and critical software defects.
3) Each risk is evaluated based on its probability and possible impact on the project.
4) High-risk functionalities are given greater testing priority to reduce the possibility of serious failures.

8) Test Reporting and Closure:

1) Test Reporting involves preparing reports that summarize testing activities, results and software quality.
2) Reports include information such as test case execution status, defect summary, requirement coverage and unresolved issues.
3) Test Closure is performed when testing objectives and defined exit criteria have been satisfied or a formal decision to stop testing has been made.
4) Final testing documents, reports and lessons learned are prepared for future reference.

Structure of Testing Group:

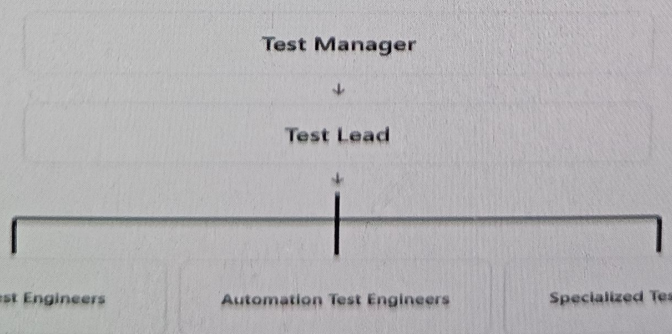


Fig: Typical Structure of a Software Testing Group

MUSA
© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

1) A Testing Group is an organized team of professionals responsible for planning, designing, executing and managing software testing activities.
2) The structure of a Testing Group defines the hierarchy, roles, responsibilities and reporting relationships among testing team members.
3) A well-organized Testing Group ensures that software testing activities are performed systematically and efficiently.
4) The structure may vary depending on the size of the organization, project complexity and testing requirements.
5) A typical Testing Group consists of a Test Manager, Test Lead, Test Engineers, Automation Test Engineers, Performance/Security Testers and Defect Management or Test Support personnel.

Q.11) Define Software Metrics. What is need of Software Metrics. List and explain different types of Software Metrics.

Ans:

1) Definition of Software Metrics

1) Software Metrics are quantitative measurements used to evaluate the characteristics of software products, software development processes and software projects.
2) They provide numerical information about quality, size, complexity, productivity, cost, progress and testing performance.
3) Examples include defect density, test coverage, execution progress, development effort and schedule variance.
4) Metrics help compare actual results with planned targets and support decisions based on measurable evidence.

2) Need for Software Metrics

1) Software metrics help measure whether the software satisfies expected quality standards and requirements.
2) They help project managers monitor development progress and identify delays at an early stage.
3) Historical metrics support realistic estimation of software effort, cost and project duration.
4) Defect-related metrics identify frequently failing modules and guide testing priorities.
5) They allow managers to evaluate the effectiveness of reviews, testing and defect removal.
6) They provide objective information for resource allocation and release-readiness decisions.
7) Metrics support continuous improvement by revealing recurring problems in development processes.
8) They help communicate project and quality status clearly to stakeholders.

3) List of Types of Software Metrics

1) Product Metrics
2) Process Metrics
3) Project Metrics

4) Explanation of Types of Software Metrics

Product Metrics

1) Product Metrics measure the characteristics and quality of the software product or its components.
2) They are used to evaluate what has been developed rather than how the development team performed its work.
3) Size metrics such as Lines of Code and Function Points indicate the approximate size of the software.
4) Complexity metrics such as Cyclomatic Complexity indicate the difficulty of program control flow.
5) Quality-related measures include defect density, reliability, maintainability and performance.
6) These metrics help testers identify complex or defect-prone modules requiring additional attention.
7) Product measurements can be collected during development and again after the software is released.
8) Example: A module containing many defects relative to its size has a high defect density and may need more testing.

Process Metrics

1) Process Metrics measure the effectiveness and efficiency of activities used to develop, review and test software.
2) They focus on the way software is produced and the performance of the development process.
3) They help identify stages where defects are introduced or where work is frequently delayed.
4) Examples include defect removal efficiency, review effectiveness and average defect resolution time.
5) Process metrics can reveal whether testing and inspections are finding defects before release.
6) They support improvement of development practices, testing procedures and team coordination.
7) Measurements collected over several projects help organizations compare process performance.
8) Example: If defects are repeatedly found late in system testing, the team may strengthen requirement and design reviews.

MUSA

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

Project Metrics

1) Project Metrics measure the planning, execution and management characteristics of a software project.
2) They help managers track whether the project is progressing according to the agreed schedule and budget.
3) Common examples include project cost, actual effort, planned effort, staffing and schedule variance.
4) They show whether available people and resources are sufficient for the remaining work.
5) Project metrics help estimate completion dates and identify activities that need additional attention.
6) They support decisions about task assignment, deadlines and resource allocation.
7) Managers use these metrics in progress reviews and reports to stakeholders.
8) Example: If actual testing effort is greater than estimated effort, the test manager may revise the schedule or assign additional resources.

Q.12) Explain with an example Incremental Integration Testing with its types.

Ans:

1) Incremental Integration Testing

1) Incremental Integration Testing is a technique in which software modules are integrated and tested step by step instead of combining all modules at once.
2) Each newly integrated module is tested with the existing modules to verify their interfaces and data exchange.
3) It is usually performed after the individual modules have completed suitable unit testing.
4) The main objective is to detect interaction defects early and make the source of failures easier to identify.
5) Stubs and drivers may be used to simulate modules that are not yet available.
6) The main types are Top-Down Integration Testing, Bottom-Up Integration Testing and Sandwich or Hybrid Integration Testing.

2) List of Types of Incremental Integration Testing

1) Top-Down Integration Testing
2) Bottom-Up Integration Testing
3) Sandwich or Hybrid Integration Testing

3) Explanation of Types

Top-Down Integration Testing

1) Testing begins with the top-level or main control module of the application.
2) Lower-level modules are added gradually according to the software control hierarchy.
3) If a lower-level module is not yet developed, a temporary stub is used in its place.
4) A stub simulates the called module and returns predefined results needed for testing.
5) After the real lower-level module becomes available, its stub is replaced and the combined system is tested again.
6) This approach allows high-level control logic and major system workflows to be examined early.
7) Interface defects between higher and lower modules can be identified at each integration step.
8) Its main limitation is that many stubs may be needed and low-level processing functions may be tested later.
9) Example: In an online shopping system, test the main order controller first and then integrate payment, inventory and notification modules one by one.

Bottom-Up Integration Testing

1) Testing begins with the lowest-level modules, such as utility functions and data-processing components.
2) Related lower-level modules are combined into small groups or clusters and tested together.
3) When a higher-level calling module is unavailable, a temporary driver is created.
4) A driver supplies input to lower modules, calls their functions and checks the returned output.
5) Once lower-level groups work correctly, they are integrated with the modules above them.
6) This approach verifies important processing and utility functions at an early stage.
7) It reduces the need for stubs because the actual lower-level modules are available for testing.
8) Its limitation is that overall control flow and complete user workflows cannot be tested until higher-level modules are integrated.
9) Example: Test tax calculation and inventory update modules first, then connect them to the order-processing controller.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

is strictly prohibited.

Sandwich or Hybrid Integration Testing

1) Sandwich Integration Testing combines the Top-Down and Bottom-Up approaches in one integration strategy.
2) Testing begins from both the upper and lower levels of the module hierarchy.
3) The upper-level modules are integrated downward while lower-level modules are integrated upward.
4) Both groups are gradually connected at an intermediate or middle layer.
5) Stubs may be used in the top-down portion and drivers may be used in the bottom-up portion.
6) This approach allows testing of high-level control logic and low-level processing functions in parallel.
7) It can reduce total integration time when teams can work on separate parts simultaneously.
8) Its limitation is that coordination, planning and management of intermediate interfaces can be more complex.
9) Example: Test the shopping interface and controller from the top while testing database and payment utilities from the bottom, then integrate both groups.

4) Example of Incremental Integration

1) Consider a system with main module A, intermediate modules B and C, and lower-level modules D, E, F and G.
2) Top-Down: Start with A and use stubs for missing modules, replace stubs with B and C, then integrate D, E, F and G.
3) Bottom-Up: Test D and E with B using a driver, test F and G with C, and finally integrate both groups with A.
4) Sandwich: Test A and upper modules downward while testing D, E, F and G upward, connect both sides at the middle layer.
5) At each integration step, check that data passed between the connected modules is correct.

Q.12) What are graph metrics? Explain with an example how to calculate cyclomatic complexity using graph metrics.

Ans:

Graph Metrics:

1) Graph Metrics are software testing measures used to analyze the structure, complexity and control flow of a software program using a graphical representation.
2) In software testing, a program is represented using a Control Flow Graph (CFG), where nodes represent statements or blocks of code and edges represent the flow of control between them.
3) Graph Metrics are mainly used in White Box Testing, particularly in Basis Path Testing, to measure the structural complexity of software.
4) They help testers identify independent execution paths, decision points and loops present in a program.
5) Graph Metrics are useful for determining the minimum number of independent paths that should be tested to achieve basis path coverage.
6) They also help developers identify complex modules that may require additional testing and maintenance.
7) One of the most important graph metrics is Cyclomatic Complexity, introduced by Thomas J. McCabe.

Cyclomatic Complexity:

1) Cyclomatic Complexity is a software metric introduced by Thomas J. McCabe to measure the logical complexity of a program.
2) It is calculated using the number of nodes, edges or decision points present in a Control Flow Graph.
3) Cyclomatic Complexity represents the number of linearly independent execution paths through a program.
4) Higher Cyclomatic Complexity indicates greater logical complexity and may require more testing effort.
5) Cyclomatic Complexity is widely used in Basis Path Testing to identify the independent paths that should be covered by test cases.

Formulas for Calculating Cyclomatic Complexity: Cyclomatic Complexity can be calculated using three methods.

Method 1: Using Edges and Nodes:

$$V(G) = E - N + 2$$

This formula applies to a connected control flow graph.

Method 2: Using Predicate Nodes:

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

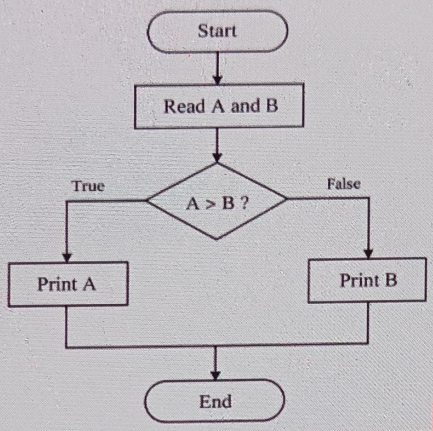
$$\boxed{V(G) = P + 1}$$

Method 3: Using Regions:

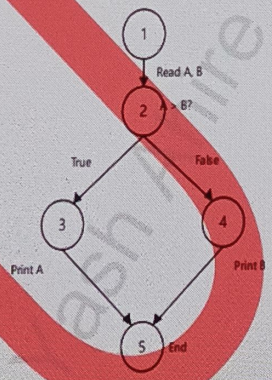
$$\boxed{V(G) = R}$$

Example: Calculation of Cyclomatic Complexity:

Consider the following program:



Control Flow Graph:



Calculation Using Graph Metrics:

Method 1: Using Edges and Nodes:

Given:

- Number of nodes, $N = 5$
- Number of edges, $E = 5$

Formula:

$$V(G) = E - N + 2$$

Substituting values:

$$V(G) = 5 - 5 + 2$$

$$\boxed{V(G) = 2}$$

Method 2: Using Predicate Nodes:

Given: Number of predicate nodes, $P = 1$

Formula:

$$V(G) = P + 1$$
$$V(G) = 1 + 1$$

$$\boxed{V(G) = 2}$$

MUSA
Path Aire
© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

Method 3: Using Regions: The Control Flow Graph contains:

- One enclosed region between the True and False branches.
- One outside region.

Therefore:

$$R = 2$$

$$\boxed{V(G) = 2}$$

Independent Paths: Since Cyclomatic Complexity is 2, the program has 2 linearly independent paths.

1) Path 1: 1 → 2 → 3 → 5 (Condition True)
2) Path 2: 1 → 2 → 4 → 5 (Condition False)

Therefore, at least two test cases are required to cover these independent paths during Basis Path Testing.

Q.14) Explain different components of Test plan.

Ans:

1) A Test Plan is a formal document that describes the objectives, scope, strategy, resources, schedule and activities required for software testing.
2) It defines what needs to be tested, how testing will be performed, who will perform the testing and when the testing activities will be completed.
3) A Test Plan is generally prepared by the Test Manager or Test Lead during the planning phase of the Software Testing Life Cycle (STLC).
4) It provides guidelines for designing test cases, executing tests, identifying defects and monitoring testing progress.
5) The main objective of a Test Plan is to ensure that software testing is performed systematically within the available time, budget and resources.
6) It also helps developers, testers and project managers understand their responsibilities and coordinate testing activities effectively.

Components of Test Plan: The following are the major components of a Test Plan:

1) Test Plan Identifier:

1) A Test Plan Identifier is a unique number or name assigned to a test plan document for identification purposes.
2) It helps distinguish the test plan from other testing documents prepared for different software projects or versions.
3) The identifier supports document tracking, version control and proper management of testing records.
4) It also helps testers and project managers locate the correct test plan during reviews and audits.

2) Test Items:

1) Test Items refer to the software components, modules or systems that are selected for testing.
2) They identify the specific parts of the application on which testing activities will be performed.
3) Test items may include individual programs, software modules, interfaces, databases or complete applications.
4) Each test item should be clearly identified along with its version or configuration, where applicable.
5) Identifying test items helps testers understand the testing scope and prevents important modules from being overlooked.
6) For example, in an online shopping application, the login module, product search module, shopping cart and payment module can be considered test items.

3) Features to be Tested:

1) This component identifies the software features and functionalities that must be verified during testing.
2) Features are selected based on software requirements, business priorities, system complexity and potential risks.
3) It includes both functional requirements and relevant non-functional requirements, such as performance, security and usability.
4) The testing team identifies the expected behavior of each selected feature before preparing test cases.
5) It helps ensure that important functionalities receive sufficient testing coverage.

4) Features Not to be Tested:

1) This component identifies the software features or functionalities that are excluded from the current testing process.
2) Some features may be excluded because they are outside the project scope, have already been tested separately or are planned for future releases.
3) The reason for excluding each feature should be clearly mentioned in the test plan.
4) It helps prevent misunderstandings between testers, developers and project stakeholders.
5) It also ensures that testing resources are concentrated on the features included in the approved testing scope.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or u is strictly prohibited.

5) Test Approach or Testing Strategy:

1) The Test Approach describes the overall methodology, techniques and procedures that will be followed during software testing.
2) It explains how the testing team will verify software requirements and identify defects.
3) It specifies the testing levels to be performed, such as Unit Testing, Integration Testing, System Testing and Acceptance Testing.
4) It identifies suitable testing techniques, including Black Box Testing, White Box Testing and Regression Testing.
5) It also determines whether testing will be performed manually, using automation tools or through a combination of both approaches.

6) Pass or Fail Criteria:

1) Pass or Fail Criteria define the conditions used to determine whether an executed test case has passed or failed.
2) A test case is considered passed when the actual output matches the expected output specified in the test case.
3) A test case is considered failed when the actual result differs from the expected result or the software does not behave according to its requirements.
4) These criteria help testers evaluate software behavior consistently and accurately.

7) Entry and Exit Criteria:

1) Entry Criteria define the conditions that must be satisfied before a particular testing phase can begin.
2) These conditions ensure that the software, testing environment and required documents are sufficiently prepared for testing.
3) Common entry criteria include approved requirements, available test cases, prepared test data and a stable software build.
4) Exit Criteria define the conditions that must be satisfied before a testing phase can be considered complete.
5) Common exit criteria include successful execution of critical test cases, achievement of required test coverage and resolution of unacceptable defects.
6) Entry and exit criteria help the testing team make systematic decisions about starting and completing testing activities.

8) Suspension Criteria and Resumption Requirements:

1) Suspension Criteria specify the conditions under which testing activities must be temporarily stopped.
2) Testing may be suspended when critical defects prevent further execution or when the testing environment becomes unavailable.
3) Other reasons may include incomplete software builds, major requirement changes or failure of essential system components.
4) Resumption Requirements define the conditions that must be satisfied before suspended testing activities can restart.
5) These conditions may include correction of critical defects, restoration of the testing environment and availability of a stable software build.

9) Testing Tasks:

1) Testing Tasks describe the individual activities that must be performed throughout the software testing process.
2) They divide the complete testing process into smaller and manageable activities.
3) Common testing tasks include requirement analysis, test planning, test case design, test environment preparation and test execution.
4) Other activities include defect reporting, defect retesting, regression testing and preparation of test summary reports.
5) Proper task identification helps the Test Manager monitor progress and ensure that testing activities are completed systematically.

10) Environmental Needs:

1) Environmental Needs describe the hardware, software, network and other infrastructure required to perform testing activities.
2) The testing environment should support the software configurations and operating conditions that need to be verified.
3) Hardware requirements may include computers, servers, mobile devices, memory and storage resources.
4) Software requirements may include operating systems, browsers, databases, application servers and testing tools.
5) The testing team may also require network connectivity, user accounts, access permissions and suitable test data.

11) Roles and Responsibilities:

1) This component defines the duties and responsibilities of individuals involved in the software testing process.
2) It identifies the Test Manager, Test Lead, Test Engineers, Automation Test Engineers and other relevant project members.
3) The Test Manager is generally responsible for planning, resource allocation, monitoring and overall coordination of testing activities.
4) The Test Lead supervises daily testing activities, distributes tasks and reviews testing progress.
5) Test Engineers design and execute test cases, identify defects and verify corrected functionality.

MUSA

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

Q.15) Explain black box and white box testing in detail. (May ask in differentiate between form also.)

Ans:

1) Black Box Testing

1) Black Box Testing is a software testing technique in which the tester checks the external behavior of software without examining its internal program code.

2) Test cases are derived from requirements, specifications, business rules and expected user behavior.

3) The tester supplies input values and compares the actual output with the expected result.

4) The main purpose is to verify that the software provides the correct functionality for valid and invalid inputs.

5) It is commonly used during functional testing, system testing and acceptance testing.

6) It helps detect missing functionality, incorrect results, interface errors and input validation problems.

7) It does not directly show which internal statements or branches have been executed.

8) Black Box Testing can be performed by testers who understand the requirements even without access to source code.

2) Black Box Testing Techniques (as per STQA Syllabus)

Boundary Value Analysis (BVA)

1) Boundary Value Analysis is a technique that selects test inputs at and near the limits of an allowed range.

2) It is based on the observation that many software defects occur at boundary conditions.

3) For an input allowed from 1 to 100, important values include 0, 1, 2, 99, 100 and 101.

4) Values just outside the allowed range help check whether invalid inputs are rejected correctly.

5) BVA is useful for age limits, marks, dates, quantities and numerical ranges.

Equivalence Class Testing

1) Equivalence Class Testing divides input data into groups expected to produce similar behavior.

2) Each group is called an equivalence class and may be valid or invalid.

3) A representative value is selected from each class instead of testing every possible value.

4) For marks from 0 to 100, classes can be less than 0, 0–100 and greater than 100.

5) This technique reduces the number of test cases while retaining representative input coverage.

State Table Based Testing

1) State-based testing checks how a system behaves in different states and how it responds to events.

2) A state table records current states, events or inputs, expected actions and next states.

3) Test cases are designed to verify both valid and invalid state transitions.

4) For example, an ATM may move between Idle, Card Inserted, Authenticated and Transaction states.

5) It is useful for systems where behavior depends on previous actions, such as login sessions and workflows.

Cause-Effect Graphing

1) Cause-Effect Graphing identifies relationships between input conditions called causes and output actions called effects.

2) The relationships are represented using logical conditions such as AND, OR and NOT.

3) A cause-effect graph helps identify important combinations of input conditions.

4) The graph can be converted into a decision table from which test cases are derived.

5) Example: A user can log in only when the username is valid AND the password is correct.

Error Guessing

1) Error Guessing is an experience-based technique in which testers predict likely errors from previous defects and system knowledge.

2) It does not follow one fixed mathematical method for choosing test inputs.

3) Testers may try blank fields, duplicate submissions, invalid formats and unexpected sequences of actions.

4) It is particularly useful for discovering defects that formal techniques may not anticipate.

5) Its effectiveness depends on the tester's experience and should complement systematic test design.

3) White Box Testing

1) White Box Testing is a technique in which test cases are designed using knowledge of the internal code, control flow and logic of the software.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

2) The tester examines how the program processes inputs and makes decisions rather than checking only external outputs.

3) The main objective is to exercise important statements, branches, paths, loops and variable uses.

4) White Box Testing is widely used in unit testing and can also support integration testing.

5) It requires access to the program structure and an understanding of programming logic.

6) It can reveal unreachable code, incorrect conditions, loop errors and some data-flow defects.

7) Coverage measures help determine which code structures have been exercised by the tests.

8) It should be combined with Black Box Testing because code coverage alone does not prove that all requirements are implemented.

4) White Box Testing Techniques (as per STQA Syllabus)

Logic Coverage Criteria

1) Logic coverage measures how thoroughly tests exercise the logical structure of a program.

2) Statement coverage checks whether executable statements are run.

3) Branch or decision coverage checks whether each decision outcome is exercised.

4) Condition coverage checks the true and false outcomes of individual Boolean conditions.

5) Higher coverage can expose untested logic, but full coverage does not guarantee defect-free software.

Basis Path Testing

1) Basis Path Testing is a structural testing technique that uses a control flow graph to identify independent execution paths.

2) Nodes represent statements or blocks and edges represent transfers of control.

3) Cyclomatic Complexity is used to estimate the number of linearly independent paths.

4) Test cases are prepared to execute the selected independent paths.

5) It helps uncover logical errors and untested branches in program control flow.

Graph Matrices

1) A Graph Matrix is a tabular representation of a program control flow graph.

2) Rows and columns represent graph nodes, while entries show connections between nodes.

3) It provides a systematic way to record and analyze possible control-flow links.

4) Weights or additional information can be attached to edges for further analysis.

5) Graph matrices support path analysis and understanding of complex program flow.

Loop Testing

1) Loop Testing focuses on checking the correct operation of loops in program code.

2) Important test cases include zero iterations, one iteration and a typical number of iterations.

3) Boundary cases near the maximum allowed number of iterations are also checked.

4) Nested and dependent loops may require additional carefully selected cases.

5) It helps detect incorrect loop conditions, infinite loops and off-by-one errors.

Data Flow Testing

1) Data Flow Testing examines the points where variables are defined, used and modified.

2) It helps detect situations such as using a variable before initialization.

3) Test cases may be designed to cover definition-use relationships along program paths.

4) The technique can reveal unnecessary assignments and incorrect variable usage.

5) It is useful when the correctness of a program depends on complex data handling.

Mutation Testing

1) Mutation Testing evaluates test effectiveness by deliberately introducing small changes into source code.

2) Each modified program is called a mutant.

3) A mutant is killed when at least one test detects behavior different from the original program.

4) Surviving non-equivalent mutants may indicate weaknesses in test cases or assertions.

5) Mutation Testing is useful for assessing test quality but can require significant execution effort.

MUSA

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

Difference Between Black Box and White Box Testing

Basis	Black Box Testing	White Box Testing
Meaning	Tests external software behavior.	Tests internal code and program logic.
Code Knowledge	Knowledge of source code is not required.	Knowledge of code structure is required.
Test Basis	Requirements and expected outputs.	Statements, conditions and control flow.
Main Focus	Functions, inputs and outputs.	Execution paths and internal operations.
Common Levels	System and acceptance testing.	Unit and selected integration testing.
Techniques	BVA, equivalence classes and state testing.	Path, loop and data-flow testing.
Defects	Missing functions and wrong results.	Logic errors and untested code paths.
Skills	Domain and requirement knowledge.	Programming and code-analysis skills.
Coverage	Functional and requirement coverage.	Statement, branch and path coverage.
Limitation	May miss hidden internal logic defects.	May miss missing requirements or incorrect specifications.

Q.16) Explain in detail Software Quality Management along with its need.

Ans:

1) Software Quality Management

1) Software Quality Management is the process of planning, assuring, controlling and improving software quality throughout the software development life cycle.

2) It aims to ensure that the software meets specified requirements, applicable standards and user expectations.

3) Software quality includes correctness, reliability, usability, maintainability, efficiency and other relevant characteristics.

4) Quality management is not limited to finding defects at the end; it includes preventing defects during development.

5) It involves developers, testers, project managers, quality specialists and other stakeholders.

6) Quality objectives and acceptance criteria should be defined before important development and testing activities.

7) Quality-related information is collected through reviews, testing, audits and metrics.

8) The organization uses results to correct problems and improve future development processes.

2) Major Activities of Software Quality Management

1) **Quality Planning:** Defines quality goals, applicable standards, procedures, responsibilities and acceptance criteria for the project.

2) **Quality Assurance:** Focuses on improving and checking the processes used to develop software, including reviews, audits and process compliance.

3) **Quality Control:** Evaluates the actual software work products through inspections, testing and defect identification.

4) **Quality Improvement:** Uses defects, feedback and measurements to remove recurring causes of quality problems and improve future performance.

3) Need for Software Quality Management

1) **Meeting User Requirements:** It helps confirm that the application performs the functions expected by users and customers.

2) **Reducing Defects:** Early reviews, standards and testing reduce the number of errors that reach later stages.

3) **Improving Reliability:** It supports consistent software operation under expected conditions.

4) **Reducing Rework Cost:** Preventing or detecting defects early generally costs less than correcting them after release.

5) **Improving Customer Satisfaction:** Reliable, usable software builds user confidence and reduces complaints.

6) **Ensuring Standards Compliance:** It helps organizations follow applicable technical, organizational or contractual standards.

7) **Managing Project Risk:** Quality monitoring helps identify critical defects and release risks before delivery.

8) **Improving Maintainability:** Consistent design and development practices make software easier to correct and enhance.

9) **Supporting Better Decisions:** Quality metrics and reports provide evidence for release and resource decisions.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

10) **Continuous Improvement:** Lessons learned from one project can improve future development and testing processes.

4) Benefits of Software Quality Management

1) Creates clear quality responsibilities and measurable expectations.

2) Encourages defect prevention instead of relying only on defect correction.

3) Improves communication between development, testing and management teams.

4) Helps deliver software that is more dependable and suitable for its intended use.

Q.17) Classify different types of bugs based on the software development lifecycle (SDLC) with an example.

Ans:

1) SDLC-Based Classification of Bugs

1) A software bug or defect is a problem in a software work product that can cause the system to behave incorrectly or fail to satisfy a requirement.

2) Bugs can be classified according to the Software Development Life Cycle stage in which they are introduced.

3) This classification helps identify the root cause of a defect and the development activity that needs improvement.

4) A defect may be introduced in an early stage but detected only during testing or after deployment.

5) Common categories include requirement, design, coding, integration, testing-related, deployment and maintenance defects.

2) Types of Bugs Based on SDLC

1) Requirement Bugs

1) Requirement bugs occur when the stated needs of users or customers are incomplete, ambiguous or incorrect.

2) They may result from misunderstanding business rules or failing to consider exceptional situations.

3) An incorrect requirement can lead to an entire feature being implemented in the wrong way.

4) Such defects should be identified through requirement reviews, clarification and validation.

5) Example: A banking application does not specify a daily transfer limit, allowing transfers beyond the intended limit.

2) Design Bugs

1) Design bugs are introduced when the software architecture, module structure, database or algorithm is planned incorrectly.

2) They can cause poor communication between modules, inefficient processing or incorrect data relationships.

3) Design defects may affect several components because many modules depend on the same architecture.

4) Design reviews, walkthroughs and interface checks help detect these problems early.

5) Example: A database design allows two customer records to share the same supposedly unique customer identifier.

3) Coding or Implementation Bugs

1) Coding bugs occur when developers incorrectly translate requirements or designs into source code.

2) They may involve wrong operators, incorrect conditions, improper loops or missing validation.

3) These defects often produce incorrect calculations, unexpected results or application crashes.

4) Unit testing, code review and static analysis can help identify them.

5) Example: A discount is added to the total price instead of being subtracted.

4) Integration or Interface Bugs

1) Integration bugs occur when separately developed modules fail to communicate or work together correctly.

2) They may result from mismatched data formats, incorrect API contracts or inconsistent assumptions.

3) Individual modules may pass unit tests but fail when combined.

4) Integration testing helps detect errors in exchanged values and module interactions.

5) Example: One module sends a date as DD/MM/YYYY while another interprets it as MM/DD/YYYY.

5) Testing-Related Bugs

1) Testing-related defects occur in test scripts, test data, expected results or testing configurations.

2) They may produce false failures or allow genuine application defects to remain undetected.

3) Incorrect expected values can cause a correct application to be reported as faulty.

4) Test case reviews and verification of automation scripts help reduce these errors.

MUSA

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

5) Example: A test script expects an old tax rate after the requirement has been updated.

6) Deployment and Configuration Bugs

1) These bugs arise when software is installed or configured incorrectly in a target environment.

2) They may involve missing dependencies, incorrect permissions, environment variables or network settings.

3) The software may work correctly in testing but fail after release because production settings differ.

4) Deployment checklists and environment verification help prevent such problems.

5) Example: The production application cannot connect to its database because the connection string is incorrect.

7) Maintenance or Change-Related Bugs

1) Maintenance bugs are introduced while correcting defects, improving features or updating existing software.

2) A change in one module may unintentionally affect another previously working module.

3) Such problems are especially common when dependencies are not fully understood.

4) Impact analysis and regression testing help detect these defects after modifications.

5) Example: A login-related bug fix unintentionally breaks the password-reset feature.

Q.18) Why is automation important in testing? What are the guidelines for selecting testing tools?

Ans:

Importance of Automation in Testing

1) Automation Testing uses tools and scripts to execute repeatable tests with limited manual intervention.

2) It saves time when the same regression tests must be executed after every code change.

3) Automated checks can provide fast feedback to developers during continuous integration.

4) Automation improves repeatability because the same test steps are followed consistently.

5) It reduces human effort for routine tasks and allows testers to spend more time on exploratory testing.

6) Automated tests can be scheduled to run outside normal working hours.

7) It helps check the application across supported browsers, devices and configurations.

8) Reports, logs and screenshots make it easier to investigate failures and track trends.

9) Reusable test scripts can support repeated releases over the lifetime of a stable application.

10) Automation can reduce long-term testing effort for suitable test cases, although initial setup and maintenance require investment.

Guidelines for Selecting Testing Tools

1) **Understand Testing Objectives:** Identify the exact testing problems the tool must solve before comparing products.

2) **Check Technology Compatibility:** Confirm support for the application type, language, operating system, browser and required environment.

3) **Select Suitable Test Cases:** Prefer stable, repeatable and high-value scenarios rather than attempting to automate every test.

4) **Evaluate Ease of Learning:** Choose tools that the team can use and maintain with available skills and training.

5) **Check Integration Capabilities:** Ensure the tool can work with version control, CI/CD, defect tracking and reporting systems.

6) **Examine Reporting Features:** The tool should produce clear execution results and useful diagnostic information.

7) **Assess Maintenance Effort:** Consider how easily test scripts can be updated when the application changes.

8) **Consider Scalability:** Check whether the tool can support larger suites and parallel execution if needed.

9) **Compare Total Cost:** Include licensing, training, infrastructure, setup, support and ongoing maintenance.

10) **Conduct a Proof of Concept:** Automate a few representative tests to evaluate reliability and practical suitability.

11) **Review Support and Documentation:** Look for reliable vendor or community support and current documentation.

12) **Avoid Tool-Driven Testing:** Choose the testing approach based on quality needs; the tool should support the strategy rather than dictate it.

Q.19) Explain the process of test suite prioritization with its type in detail with example.

Ans:

Test Suite Prioritization

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

1) Test Suite Prioritization is the process of arranging test cases in an order so that important testing objectives can be achieved earlier.

2) It is particularly useful when the available time is insufficient to execute every test immediately.

3) The goal may be early fault detection, rapid coverage of important requirements or faster feedback on changed software.

4) Prioritization changes the execution order of tests; it does not necessarily remove tests from the suite.

5) Test cases can be ranked using risk, past defect history, coverage, business importance or execution time.

6) It is commonly applied to regression testing after changes to a software application.

List of Types of Test Suite Prioritization

1) General Test Case Prioritization

2) Version-Specific Test Case Prioritization

Explanation of Types of Test Suite Prioritization

1) General Test Case Prioritization

1) General Test Case Prioritization arranges tests to be useful across multiple future versions of the software.

2) It does not depend on knowing the exact code changes in a particular new release.

3) The priority order may be based on general requirement importance, test coverage and previous defect history.

4) Tests covering business-critical functions are usually assigned higher priority.

5) A test that has detected many serious defects in the past may be placed earlier in the order.

6) The same ordering can be reused for several releases, although it should be reviewed periodically.

7) It is useful when detailed change information is unavailable or the team needs a stable regression testing order.

8) Its limitation is that it may not give the highest priority to tests affected by the most recent code changes.

9) Example: For an online banking application, login, money transfer and balance checking may consistently be tested before profile color settings.

2) Version-Specific Test Case Prioritization

1) Version-Specific Test Case Prioritization arranges tests for a particular modified version of the software.

2) It uses information about the changes introduced in the current build or release.

3) The testing team identifies changed modules, affected interfaces and related requirements.

4) Tests directly related to modified or high-risk components are assigned higher priority.

5) Historical defects, code coverage and business impact can also be considered in the ranking.

6) The order is recalculated whenever a new version contains a different set of changes.

7) It helps detect regression defects introduced by the current modifications at an early stage.

8) Its limitation is that it requires accurate change information and additional analysis for each version.

9) Example: If a new release changes the payment module, payment and order-confirmation tests may be executed before login appearance tests.

Process of Test Suite Prioritization

1) Identify the existing test suite and the reason prioritization is needed.

2) Define the objective, such as detecting critical defects as early as possible.

3) Collect information about business risks, requirements, previous defects and recent code changes.

4) Choose an appropriate prioritization approach and assign a priority to each test case.

5) Arrange tests from highest priority to lowest priority.

6) Execute tests in the selected order and record their results.

7) Review defect detection and coverage achieved, then revise priorities for future testing.

Example of Test Suite Prioritization

1) Consider an online shopping application with tests for payment, login, order confirmation, profile photo and theme change.

2) Using general prioritization, payment and login tests are placed first because they are consistently important to customers.

3) Suppose a new version changes only the profile photo upload feature and its image storage service.

4) Using version-specific prioritization, profile upload and related storage tests may be moved earlier because they are directly affected by the latest change.

MUSA

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or u
is strictly prohibited.

5) Thus, general prioritization uses long-term importance, while version-specific prioritization responds to the current release.

Q.20) Explain software testing lifecycle. Explain software testing model with a neat diagram.

Ans:

1) Software Testing Life Cycle (STLC) is a systematic process consisting of different phases performed to verify and validate the quality of a software application.
2) It defines the sequence of testing activities, starting from requirement analysis and continuing until test closure.
3) The main objective of STLC is to identify software defects, verify requirements and ensure that the application performs according to expected specifications.
4) Each phase of STLC has specific objectives, activities, deliverables and entry and exit criteria.

Phases of Software Testing Life Cycle: STLC generally consists of six major phases.

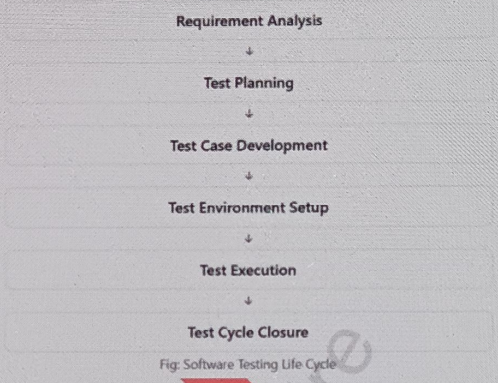


Fig: Software Testing Life Cycle

1) Requirement Analysis:

1) Requirement Analysis is the first phase of STLC, in which the testing team studies the software requirements to understand the expected functionality of the application.
2) Testers review the Software Requirement Specification (SRS), functional requirements and relevant non-functional requirements.
3) The main objective is to identify testable requirements and determine which functionalities require verification.
4) Testers communicate with developers, business analysts and stakeholders to clarify incomplete or ambiguous requirements.
5) The testing team identifies the types of testing required, such as functional testing, performance testing and security testing.

2) Test Planning:

1) Test Planning is the phase in which the testing team prepares a detailed plan for performing software testing activities.
2) The Test Manager or Test Lead defines the testing objectives, scope, strategy and overall testing approach.
3) The team estimates the time, effort, budget and resources required to complete testing.
4) Suitable testing techniques, tools and testing environments are identified according to project requirements.
5) Testing tasks are assigned to team members, and a schedule is prepared for test design, execution and reporting.
6) Entry criteria, exit criteria, risks and test deliverables are also defined during this phase.

3) Test Case Development:

1) Test Case Development involves designing and preparing test cases based on the software requirements and testing objectives.
2) Each test case specifies the input data, execution steps, expected results and conditions required to verify a particular functionality.
3) Testers prepare positive and negative test cases to check both valid and invalid software behavior.
4) Appropriate test design techniques, such as Equivalence Partitioning, Boundary Value Analysis and Decision Table Testing, may be used.
5) Required test data is identified and prepared for executing the test cases.
6) Test cases are reviewed to ensure correctness, completeness and sufficient requirement coverage.

4) Test Environment Setup:

1) Test Environment Setup involves preparing the hardware, software and infrastructure required to execute test cases.

MUSA
Kashish
© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

2) The testing team identifies the operating systems, browsers, databases, servers and network configurations needed for testing.
3) The application build is installed and configured in the testing environment.
4) Required testing tools, user accounts, access permissions and test data are made available.
5) The environment should represent the intended operating conditions sufficiently to produce meaningful testing results.
6) Smoke Testing or basic environment verification may be performed to confirm that the application is stable enough for detailed testing.

5) Test Execution:

1) Test Execution is the phase in which testers execute the prepared test cases against the software application.
2) Actual results obtained during execution are compared with the expected results specified in the test cases.
3) A test case is marked as passed when the actual result matches the expected result and failed when it does not.
4) Defects identified during testing are recorded in defect tracking tools such as JIRA or Bugzilla.
5) Reported defects are assigned to developers for investigation and correction.
6) Test execution results, defect status and requirement coverage are monitored throughout this phase.

6) Test Cycle Closure:

1) Test Cycle Closure is the final phase of STLC, in which completed testing activities and results are reviewed.
2) The testing team checks whether the defined testing objectives and exit criteria have been satisfied.
3) Test execution reports are analyzed to determine the number of passed, failed and blocked test cases.
4) Defect reports are reviewed to identify resolved defects, remaining defects and associated risks.
5) A Test Summary Report is prepared to provide an overview of testing activities and software quality.

Q.21) Explain the goal of software testing. Explain the verification of requirements and objectives in Software testing.

Ans:

1) Goals of Software Testing

1) One major goal of software testing is to identify defects by executing or evaluating software and its related work products.
2) Testing checks whether the application satisfies specified functional requirements and expected business behavior.
3) It helps assess important quality characteristics such as reliability, usability, security and performance.
4) Testing provides information about product quality and remaining risks to developers, managers and customers.
5) It verifies that important inputs, outputs, calculations and interfaces behave as expected.
6) It checks whether software handles invalid data, boundary conditions and unexpected situations appropriately.
7) Early testing and reviews help reduce the cost of correcting defects at later development stages.
8) Testing supports release decisions by showing what was tested, which tests passed and which defects remain.
9) It builds confidence in software behavior, although testing cannot prove that the application contains no defects.
10) Testing also helps prevent future defects by revealing weaknesses in requirements, design and development practices.

2) Verification of Requirements

1) Requirements verification checks whether documented requirements are clear, correct, complete, consistent and suitable for design and testing.
2) It is generally performed before coding through requirement reviews, inspections and discussions with stakeholders.
3) Each requirement should be understandable and should avoid vague expressions such as "fast" or "user-friendly" without measurable criteria.
4) Requirements should be checked for completeness so that important functions, constraints, interfaces and error conditions are not omitted.
5) Consistency checking identifies requirements that contradict one another or conflict with project constraints.
6) Feasibility checking determines whether requirements can be implemented using available technology, time and resources.
7) Traceability checking links each requirement to its source, design elements and corresponding test cases.
8) Testability checking ensures that the expected result can be observed and verified through an appropriate test.
9) Stakeholder reviews help confirm that the requirement specification reflects actual user and business needs.
10) Problems found during verification should be corrected before they affect system design, coding or testing.

3) Objectives of Requirements Verification

1) Identify missing, ambiguous and contradictory requirements at an early stage.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

2) Ensure that each requirement is measurable and can be tested.

3) Reduce misunderstandings between customers, analysts, developers and testers.

4) Provide a reliable foundation for design, implementation and test-case preparation.

5) Improve traceability from user needs to testing evidence.

6) Reduce rework and project risk caused by incorrect requirements.

Q.22) Explain different Quality aspects that need to be considered while testing web based system.

Ans:

1) Web-Based System Testing

1) Web-Based System Testing checks the quality of an application accessed through web browsers and network connections.

2) Web applications may be used on different devices, browsers and networks, so testing must consider both functionality and operating environment.

3) Quality testing should cover user-visible behavior, server-side processing, data handling and communication between components.

2) Quality Aspects of Web-Based Systems

Functionality

1) Check that each web page, form, link, button and feature performs the required operation.

2) Verify correct handling of valid, invalid and missing user inputs.

3) Confirm that business rules and calculations produce expected results.

4) Check error messages and recovery behavior when operations fail.

Usability

1) Ensure that users can understand the navigation, labels, instructions and workflows.

2) Check whether important tasks can be completed with reasonable effort.

3) Verify that feedback and validation messages are clear and helpful.

4) Assess readability and accessibility for intended users.

Performance

1) Measure page response time and transaction completion time under expected conditions.

2) Check application behavior when many users access it simultaneously.

3) Observe server resource usage, slow queries and bottlenecks.

4) Test performance under varying network speeds and peak loads.

Security

1) Verify authentication, authorization and session handling.

2) Check that sensitive information is protected during storage and transmission.

3) Test input validation and protections against common web attacks.

4) Ensure that users cannot access data or functions without permission.

Compatibility

1) Check important workflows in supported browsers and browser versions.

2) Verify display and functionality on supported desktop, tablet and mobile devices.

3) Test differences in operating systems, screen sizes and resolutions.

4) Ensure that changes in browser behavior do not break important features.

Reliability and Availability

1) Check that the system continues to function during normal and prolonged use.

2) Observe behavior during network interruptions, server errors and unexpected failures.

3) Verify that recovery mechanisms protect important data and transactions.

4) Assess whether the service meets its availability expectations.

Interface and Navigation

1) Check that links lead to correct pages and that menus behave consistently.

MUSA

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or u
is strictly prohibited.

2) Verify the exchange of data between browser, server, APIs and databases.

3) Check that page layouts and navigation remain understandable.

4) Ensure that error pages and redirects are handled appropriately.

Data Integrity

1) Confirm that submitted information is stored and retrieved correctly.

2) Check that concurrent operations do not produce inconsistent data.

3) Verify transaction completion, rollback and duplicate-submission handling.

4) Test input constraints and database validation rules.

Q.23) How do Unit Testing and Integration Testing differ from each other?

Ans:

1) Unit Testing

1) Unit Testing checks the smallest testable components of software, such as functions, methods or classes.

2) It is usually performed during development before the components are integrated.

3) Its main objective is to find defects in individual component logic and behavior.

4) Mocks, stubs or other substitutes may be used to isolate the component from its dependencies.

5) Unit tests are generally fast and are often automated.

2) Integration Testing

1) Integration Testing checks whether two or more combined modules work together correctly.

2) It focuses on interfaces, data transfer, communication and interactions between components.

3) It is performed after the relevant individual modules are sufficiently tested.

4) It may follow top-down, bottom-up, sandwich or other planned integration approaches.

5) It helps detect interface mismatches and errors that are not visible when modules are tested separately.

3) Difference Between Unit Testing and Integration Testing

Basis	Unit Testing	Integration Testing
Meaning	Tests individual software components.	Tests combined components and their interactions.
Main Focus	Internal behavior of one unit.	Interfaces and data flow among units.
Test Object	Function, method, class or small module.	Two or more connected modules or subsystems.
Execution Stage	Usually begins during coding.	Usually follows relevant unit testing.
Defects Found	Logic, calculations and local validation errors.	Interface, communication and data exchange errors.
Dependencies	Often isolated with mocks or stubs.	Requires interacting components or substitutes.
Test Environment	Usually simple and controlled.	May need additional integrated services and data.
Test Speed	Generally quick to execute.	Can be slower because multiple parts interact.
Typical Responsibility	Often written by developers.	Performed by developers and/or testing teams.
Example	Checking one tax calculation function.	Checking order module interaction with payment module.

Q.24) Differentiate between the following:

Ans:

i) Effective Software Testing vs Exhaustive Software Testing

Effective Software Testing	Exhaustive Software Testing
1) Effective software testing selects a practical set of tests that can reveal important defects with available time and resources.	1) Exhaustive software testing attempts to check every possible input, condition, combination and execution path.
2) Its main objective is to find significant failures efficiently and provide useful evidence about software quality.	2) Its main objective is to cover all possible situations so that no potential test combination is left unchecked.
3) Test cases are selected according to requirements, risk, boundary conditions and likely failure areas.	3) Test cases would have to include every possible value and combination, even when many are similar.
4) Techniques such as equivalence partitioning and boundary value analysis help reduce unnecessary test cases.	4) Reducing the number of possible test cases would conflict with the idea of testing every combination.

MUSA

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

5) It is practical for real projects because it works within testing schedules, budgets and available staff.	5) It is generally impractical for real software because inputs, states and execution paths can be extremely numerous.
6) It prioritizes business-critical functionality and areas where failures could cause serious problems.	6) It gives complete coverage priority, regardless of whether some cases have lower business impact.
7) It aims for sufficient requirement and risk coverage rather than claiming that every defect has been found.	7) It aims for complete possible-case coverage, which is rarely achievable for nontrivial applications.
8) Testing effort can be adjusted when software requirements or release deadlines change.	8) The amount of work grows rapidly when new inputs, features or combinations are introduced.
9) Example: For an age field accepting 18–60, test representative valid values and boundaries such as 17, 18, 60 and 61.	9) Example: For the same age field, test every possible entered value and input form, including all relevant character sequences.
10) Its limitation is that some defects may remain because only selected cases are executed.	10) Its limitation is the enormous execution time, cost and combinations involved, making it unsuitable as a general strategy.

ii) Traditional Software Testing vs Web-Based Software Testing

Traditional Software Testing	Web-Based Software Testing
1) Traditionally installed applications are often tested in a relatively controlled client or desktop environment.	1) Web-based applications are tested across browsers, servers, networks and multiple user environments.
2) The application may be installed and executed locally on the user's computer.	2) The application is generally accessed through a browser and communicates with a remote server.
3) Testing focuses on application functions, local resources and supported operating systems.	3) Testing also emphasizes links, forms, browser behavior, APIs, sessions and server-side processing.
4) Compatibility testing mainly considers supported platforms and installed application versions.	4) Compatibility testing must consider browsers, browser versions, screen sizes and devices.
5) Many standalone applications can continue working without an active network connection.	5) Many web applications depend on network availability, latency and server connectivity.
6) Performance testing checks local processing, memory usage and application response.	6) Performance testing also checks page loading, server response, concurrent users and network delays.
7) Security testing focuses on the application, its local data and its operating environment.	7) Security testing additionally examines web authentication, session management and common web attacks.
8) Installation and version upgrades may be required on individual user devices.	8) Many application updates can be deployed centrally on servers without reinstalling the browser client.
9) The user interface may be designed for a particular desktop or installed platform.	9) The interface often needs responsive layouts and consistent behavior across different devices.
10) Example: Testing an offline desktop billing application on supported Windows versions.	10) Example: Testing an online shopping website on different browsers, phones and network conditions.

iii) Primary Mutants vs Secondary Mutants

Primary Mutants	Secondary Mutants
1) A primary mutant is a modified program produced by applying one mutation change to the original code.	1) A secondary mutant is produced by combining two or more mutation changes in a program.
2) It usually represents one artificial programming fault at a time.	2) It represents multiple artificial faults that may interact with each other.
3) Only one operator, constant, condition or statement is modified in a single mutant.	3) More than one operator, constant, condition or statement may be modified.
4) Its changed behavior is generally easier to trace to the single mutation introduced.	4) Its behavior can be harder to understand because several changes may influence the result.
5) It is commonly used in first-order mutation testing.	5) It is associated with higher-order mutation testing.
6) Primary mutants are useful for checking whether tests detect individual small faults.	6) Secondary mutants are useful for studying whether tests detect combinations of faults.
7) Generating one mutation per mutant is relatively straightforward.	7) Combining multiple mutations can greatly increase the number of possible mutants.
8) A surviving primary mutant may indicate that the test suite does not detect a particular single change.	8) A surviving secondary mutant may result from missed faults or interactions that hide the effects of changes.
9) Example: Replace the comparison `x > 5` with `x >= 5`.	9) Example: Replace `x > 5` with `x >= 5` and also replace an addition with subtraction.
10) Analysis is usually simpler, although equivalent mutants may still occur.	10) Analysis is more complex, and combinations of changes may cancel or mask one another.

iv) Static Testing vs Dynamic Testing

Static Testing	Dynamic Testing
1) Static testing examines software work products without executing the program.	1) Dynamic testing evaluates software behavior by executing the program.

MUSA

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

2) It can be applied to requirements, design documents, source code and test cases.	2) It is applied to executable modules, integrated systems and working applications.
3) Common activities include reviews, walkthroughs, inspections and static code analysis.	3) Common activities include unit, integration, system and acceptance test execution.
4) It can begin early, even before any executable code is available.	4) It requires executable software or a testable software component.
5) It finds problems such as ambiguous requirements, design inconsistencies and suspicious code patterns.	5) It reveals runtime failures such as wrong outputs, crashes and incorrect behavior.
6) The work product is checked against standards, specifications and expected structure.	6) Actual results are compared with expected results using test inputs.
7) Static analysis tools may identify possible issues without running the application.	7) Automated test tools may execute test scripts and record pass or fail results.
8) It helps prevent defects from reaching later stages of development.	8) It helps detect failures that become visible when software actually runs.
9) Example: Reviewing a requirement to detect missing acceptance criteria.	9) Example: Executing a login form with an incorrect password to check the response.
10) It cannot by itself confirm actual runtime behavior.	10) It may not identify missing or ambiguous requirements unless corresponding tests were designed.

v) Verification vs Validation

Verification	Validation
1) Verification checks whether software work products conform to specified requirements and design rules.	1) Validation checks whether the resulting software satisfies actual user needs and intended use.
2) It answers the question, "Are we building the product correctly?"	2) It answers the question, "Are we building the right product?"
3) Its main focus is correctness and consistency with the documented specification.	3) Its main focus is whether the software is suitable and useful for its intended purpose.
4) It may examine requirements, architecture, detailed designs, code and supporting documents.	4) It commonly evaluates the implemented application and its user-facing behavior.
5) Reviews, inspections, walkthroughs and analyses are common verification activities.	5) System testing, user acceptance testing and realistic usage evaluation support validation.
6) It helps find missing design details, inconsistencies and departures from requirements.	6) It helps identify features that do not meet user expectations or solve the intended problem.
7) It can be performed before the software is executable.	7) It often involves executing working software, although validation activities can begin earlier.
8) Developers, analysts, designers and reviewers may carry out verification.	8) Testers, product owners, customers and representative users may participate in validation.
9) Example: Checking whether a password-reset design meets the approved requirements.	9) Example: Checking whether real users can successfully recover their accounts.
10) Passing verification does not guarantee that the requirements themselves meet user needs.	10) Validation provides evidence that the finished product is fit for its intended use.

Q.25) Write short notes on the following:

i) Six Sigma

Ans:

1) Six Sigma is a systematic, data-driven quality improvement methodology used to reduce process variation, defects and errors.

2) It focuses on improving the consistency and quality of products or services by studying how a process actually performs.

3) The term sigma (σ) represents standard deviation, a statistical measure of variation in a process.

4) A higher sigma quality level generally indicates fewer defects relative to the opportunities for defects.

5) Six Sigma is commonly associated with the benchmark of approximately 3.4 defects per million opportunities under its conventional assumptions.

6) The methodology focuses on understanding customer requirements and identifying activities that create defects.

7) It relies on measurement and statistical analysis rather than assumptions or guesswork.

8) One widely used Six Sigma improvement cycle is DMAIC: Define, Measure, Analyze, Improve and Control.

9) In the Define phase, the organization identifies the problem, improvement goals and customer expectations.

10) In the Measure phase, it collects reliable information about current process performance and defects.

11) In the Analyze phase, it investigates the main causes of errors, variation and process inefficiency.

12) In the Improve phase, the organization introduces solutions to reduce or eliminate those causes.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or u is strictly prohibited.

13) In the Control phase, the improved process is monitored so that the same problems do not return.

14) Six Sigma encourages continuous improvement, measurable results and involvement of responsible team members.

Application of Six Sigma in Software Development

1) It helps software teams identify frequently occurring defects and determine their root causes.

2) It can improve requirement gathering by measuring and reducing errors caused by unclear or incomplete specifications.

3) It supports improvement of coding and review processes to reduce the introduction of defects.

4) Software testing teams can analyze defect density, defect leakage and rework effort to identify weak areas.

5) It helps improve defect resolution processes by reducing delays and repeated reopening of bugs.

6) Teams can use DMAIC to investigate problems such as high production failure rates or frequent release delays.

7) Process measurements help compare performance before and after an improvement.

8) It encourages standard procedures for development, code review, testing and defect reporting.

9) Reducing avoidable rework can improve productivity, cost control and customer satisfaction.

10) Example: A team measures repeated login failures, identifies a faulty validation process, improves its checks and monitors later releases for recurrence.

ii) ISO 9000:2000:

Ans:

1) ISO 9000:2000 is an edition of the ISO 9000 standard that describes the fundamentals and vocabulary of quality management systems (QMS).

2) It belongs to the ISO 9000 family of international standards developed to support consistent quality management.

3) Its purpose is to help organizations understand the concepts, principles and terminology used in quality management.

4) It emphasizes meeting customer requirements and improving customer satisfaction through effective processes.

5) The standard describes quality management as a systematic organizational activity rather than only final-product inspection.

6) It supports the process approach, in which related activities are identified, managed and improved as processes.

7) It emphasizes the importance of leadership in establishing quality objectives and organizational direction.

8) It encourages the involvement of people because quality depends on employees at different levels.

9) It supports a system approach to managing interrelated processes within an organization.

10) Continual improvement is an important quality management principle, helping organizations improve performance over time.

11) Decisions should be based on reliable information and analysis rather than unsupported assumptions.

12) It encourages mutually beneficial relationships with suppliers and other relevant external parties.

13) In software development, these principles can guide requirements management, development procedures, testing and defect control.

14) Documentation, defined responsibilities and measurement help organizations apply quality management consistently.

15) ISO 9000:2000 provides concepts and vocabulary; ISO 9001:2000, a related standard, specified QMS requirements used for certification.

16) The 2000 edition is historical and is discussed here because it is the version specified in the examination syllabus.

iii) Efficient Test Suite Management:

Ans:

1) Efficient Test Suite Management is the process of organizing, selecting, maintaining and executing test cases so that testing objectives are achieved with reasonable effort.

2) A test suite is a collection of test cases designed to check the behavior and quality of software.

3) As a project grows, the test suite may become large because new tests are added for features and defect fixes.

4) Some test cases may become outdated, duplicated or unnecessarily repetitive.

5) Efficient management involves reviewing test cases regularly and updating them when requirements change.

6) It includes identifying critical tests and prioritizing them according to business risk and testing objectives.

7) It involves maintaining test data, expected results and traceability between tests and requirements.

8) Automation may be used for suitable repeatable tests to reduce manual execution effort.

9) Test results and historical defect information help teams identify useful and less useful tests.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

10) Test suite minimization is one technique used in efficient test suite management.

11) Test suite minimization selects a smaller set of test cases while preserving specified coverage requirements.

12) The main aim is to remove redundant tests without unnecessarily weakening important testing objectives.

Benefits of Test Suite Minimization

1) It reduces the total number of test cases that must be executed during repeated testing.

2) It saves testing time, especially when regression testing is performed frequently.

3) It can reduce the cost of test infrastructure and execution resources.

4) It helps teams obtain test results and feedback more quickly.

5) It reduces the effort required to maintain duplicated or overlapping test cases.

6) It makes the test suite easier to organize, understand and review.

7) It can improve the efficiency of continuous integration and frequent software releases.

8) It allows testers to concentrate on important tests and high-risk application features.

9) It helps reduce unnecessary repetition while preserving the chosen coverage objectives.

10) However, minimizing a suite may reduce defect-detection ability even when coverage is preserved, so tests should be removed carefully.

iv) Need for Automation in Testing:

Ans:

1) Automation testing uses tools and scripts to execute repeatable tests with limited manual effort.

2) It reduces time spent repeating the same regression tests after software changes.

3) Automated checks can run frequently and provide early feedback to developers.

4) It supports continuous integration by checking new software builds automatically.

5) It improves consistency because the same steps and expected results are used on repeated executions.

6) It helps test supported browser, device and operating-system combinations.

7) Automated execution can be scheduled outside working hours.

8) It produces logs and reports that support defect investigation.

9) It allows manual testers to spend more time on exploratory, usability and unusual test scenarios.

10) It can reduce long-term execution effort for stable, high-value test cases.

11) Automation requires initial investment in tools, scripts, environments and training.

12) Automated scripts must be maintained when the application or requirements change.

v) JIRA:

Ans:

1) JIRA is a project and issue-tracking platform widely used in software development and testing teams.

2) It is used to organize tasks, report software defects and monitor their progress through defined workflows.

3) Testers can create defect issues with a title, description, reproduction steps and expected and actual results.

4) Each issue can be assigned to a responsible person and given a priority and status.

5) JIRA helps developers and testers communicate about defects through comments and issue updates.

6) It supports planning and monitoring work in Agile teams, including Scrum and Kanban projects.

7) Teams can link defects with user stories, tasks and related development work.

8) JIRA is primarily an issue and project management tool; dedicated test execution often uses integrations or additional test management products.

Features of JIRA

1) Issue Creation and Tracking: Records defects, tasks and requests with detailed information.

2) Custom Workflows: Allows teams to define statuses and transitions for issue resolution.

3) Priority and Assignment: Helps identify urgent defects and assign responsibility.

4) Boards and Backlogs: Supports Agile planning and monitoring of work.

5) Dashboards and Reports: Displays issue counts, progress and project status.

6) Search and Filters: Helps locate issues by project, status, assignee and other fields.

7) Comments and Notifications: Improves communication among developers, testers and managers.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or u
is strictly prohibited.

2) Custom Workflows: Allows

3) Priority and Assignment: Helps identify urgent defects and assign responsibility.

4) Boards and Backlogs: Supports Agile planning and monitoring of work.

5) Dashboards and Reports: Displays issue counts, progress and project status.

6) Search and Filters: Helps locate issues by project, status, assignee and other fields.

7) Comments and Notifications: Improves communication among developers, testers and managers.

© 2024 MUSA. All rights reserved. This material is the exclusive property of MUSA. Unauthorized reproduction, distribution, or use is strictly prohibited.

8) Integration Support: Connects with development, automation and test management tools.

9) Attachments and Links: Supports screenshots, evidence and relationships between issues.

Bugzilla:

1) Bugzilla is an open-source, web-based defect tracking system used to report and manage software bugs.

2) It helps testers document problems found during software testing.

3) Each defect report contains information such as the problem description, steps to reproduce and expected behavior.

4) Developers use Bugzilla to review, investigate, fix and update the status of reported defects.

5) Bugzilla helps teams assign responsibility and prioritize defects according to severity and importance.

6) It maintains the history of changes made to defect records.

7) It supports collaboration between testers, developers and project managers.

8) Bugzilla mainly manages bugs and does not itself serve as a general automated test execution engine.

Features of Bugzilla

1) Bug Reporting: Creates structured records containing defect descriptions and reproduction steps.

2) Defect Status Tracking: Monitors issues from reporting to resolution and closure.

3) Severity and Priority: Classifies defects by impact and urgency.

4) Assignment: Allocates defects to the responsible developer or team member.

5) Advanced Search: Finds defects using conditions, filters and saved searches.

6) Comments and Attachments: Allows discussion and supporting screenshots or logs.

7) Email Notifications: Informs users when relevant bug records change.

8) History Tracking: Maintains a record of modifications to each issue.

9) Reports and Charts: Helps review defect trends and project quality.

*****ALL THE BEST*****

MUSA